
Computeroefenzitting 0

Introductie tot R

Bachelor in de Informatica: **Elementaire statistiek**

1 Downloaden van de *R* en *RStudio* software

Doorheen de computeroefenzittingen gebruiken we het software programma *R* in combinatie met het software programma *RStudio*. Beide zijn gratis programma's. *R* kan je downloaden via:

- Windows: <http://cran.freestatistics.org/bin/windows/base/>
- Mac: <http://cran.r-project.org/bin/macosx/>
- Linux: <http://cran.r-project.org/bin/linux/>

Hoewel *R* voldoende is, wordt aangeraden om ook de *RStudio* interface te downloaden. Deze interface biedt extra mogelijkheden en de grafische interface lijkt op die van *Matlab*. *RStudio* kan je downloaden via <http://www.rstudio.com/products/rstudio/download/>. Kies dan de versie voor jouw besturingssysteem.

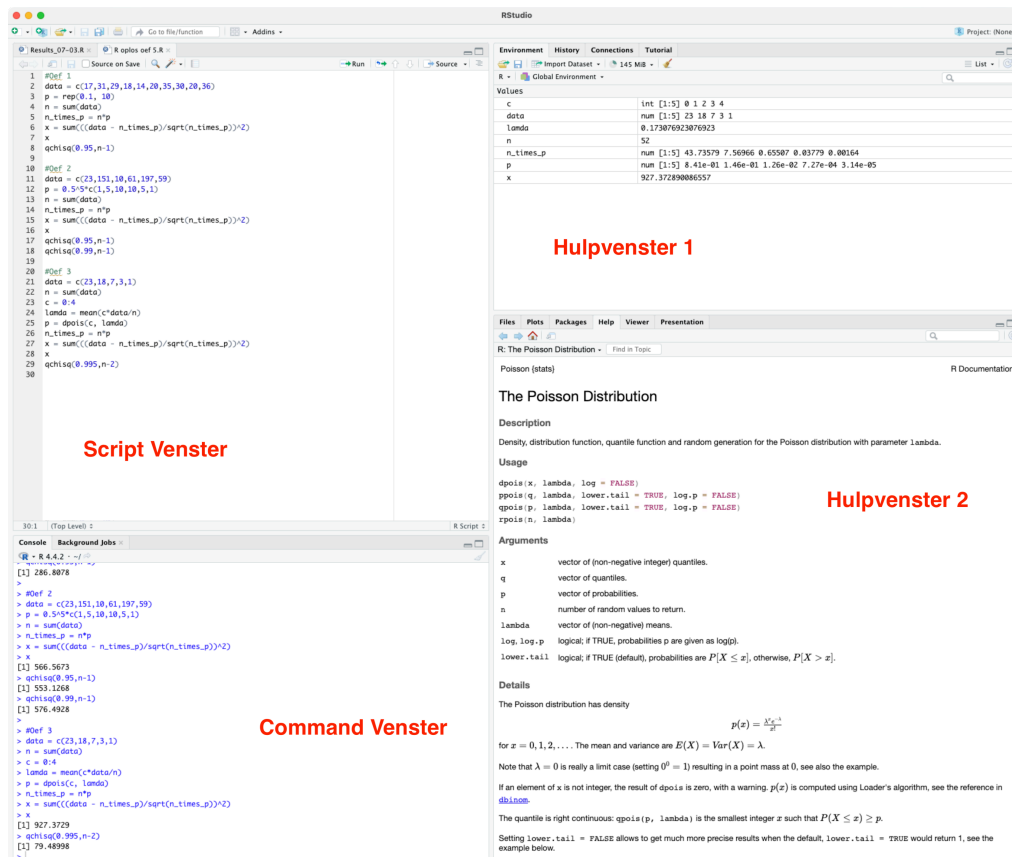
In deze (en volgende) computeroefenzittingen zal er met *RStudio* gewerkt worden.

2 Werken met *R* via *RStudio*

RStudio werkt met vier basisvensters, deze worden getoond in Figuur 1.

- Command venster of console vind je links onder. In dit venster worden alle commando's uitgevoerd. Het is echter inefficiënt om code rechtstreeks in dit venster in te geven. Bovendien kan je nadien je code dan niet opnieuw uitvoeren. Om dit te vermijden maken we aparte bestanden, **scripts** genaamd, waarin we de commando's schrijven en dan zullen uitvoeren in de console. Dit doen we in een ander venster.
- Script venster. Indien dit venster niet geopend werd bij het starten van *RStudio*, ga je linksboven naar *File>New> R script*. Je krijgt nu een leeg bestand. In dit bestand kan je code invoeren en vervolgens opslaan via *File>Save* zodat je ze later eenvoudig opnieuw kan hergebruiken.
- Hulpvenster 1. Dit venster heeft twee tabs: "Environment" en "History". Onder de tab History vind je de vorige commando's terug die in de console werden ingegeven. Je kan er een commando aanklikken en het naar de console sturen via "To Console" of naar een script via "To Source". Onder de tab Environment worden alle variabelen, functies,... weergegeven die momenteel in het geheugen aanwezig zijn. Via deze tab zullen we later ook datasets inlezen. Het bezemsteel-icoontje maakt alle bekende variabelen, functies,... leeg.
- Hulpvenster 2. Dit venster heeft vijf tabs. Via "Files" kan je naar bestanden browsen en deze openen. Onder de tab "Plots" worden figuren getoond. Je kan vanuit deze tab een figuur groter maken via "Zoom" of opslaan via "Export". Indien je meerdere figuren maakte kan je door deze figuren bladeren met de pijl icoontjes. Het tabblad "Help" kan nuttig zijn om dingen op te zoeken. Je vindt er ook de "An Introduction to R" handleiding waar je veel informatie over *R* kan vinden. In de tab "Packages" beheer je alle packages

(of libraries) die gebruikt of wilt toevoegen. Het tabblad “Viewer” kan gebruikt worden om lokale webtoepassingen te bekijken maar dit valt buiten het bereik van deze inleiding.



Figuur 1: Het scriptvenster, het command venster en de twee hulpvensters in *RStudio*.

Open een script, indien je dit nog niet gedaan hebt, en typ volgende commando's:

```
5+3
x <- 5+3
y = 9-2
x
x*y
```

Om code vanuit je script uit te voeren, ga je op een lijn met code staan en druk je *Ctrl+Enter* of klik je op de knop “Run”. Voer op deze manier het commando `5+3` uit. Merk op dat in de console het commando en het resultaat verschijnt. Om meerdere lijnen tegelijk uit te voeren kan je verschillende lijnen tegelijk selecteren en vervolgens op *Ctrl+Enter* te drukken. Voer nu de overige lijnen uit. Merk op dat in de console de waarde van `x` wordt weergegeven en het resultaat van de bewerking `x * y`. In hulpvenster 1 zie je bij Environment dat de variabelen `x` en `y` de waarden 8 en 7 toegewezen krijgen. Je kan dus aan een variabele een waarde toekennen via `<-` of `=`.

Let op: gebruik nooit de letters `c` en `t` voor namen van variabelen, vermits dit tevens namen van functies zijn en er hierdoor conflicten kunnen ontstaan. Vanzelfsprekend vermijd je ook best andere functienamen zoals `log`, `exp`, ...

De variabelen `x` en `y` hebben nu een waarde die is opgeslagen in het geheugen. Het zou kunnen dat je deze variabelen de volgende keer dat je wil werken opnieuw wil gebruiken. Om de waarden van deze variabelen op te slaan, ga je naar *Session>Save Workspace As*. Dit menu vind je

bovenaan je scherm. Je kan dan later de Workspace terug laden via *Session>Load Workspace*. De waarden van x en y zullen dan terug in het geheugen worden geladen.

Voor meer informatie over hoe een commando juist werkt, ga je naar het tabblad “Help” in hulpvenster 2. Hier kan je info zoeken over een commando of een bepaalde term. Zoek bijvoorbeeld info over de logaritme. Als je meer info wil, ga je naar *Help>R Help* voor informatie over *R*. Informatie over *RStudio* vind je via *Help>About RStudio* en via *Help>Keyboard Shortcuts* vind je een lijst van alle sneltoetsen.

Voer de volgende code uit in *RStudio*:

```
z <- log(x)
?log
a <- sqrt(x)*z^2
b <- exp(a+z)
rm(x,y) #Removes x and y from environment
ls() #Shows environment
rm(list=ls()) #Clear the whole environment
```

Merk op dat het teken ? dus ook gebruikt kan worden om de help pagina van dat commando te openen. Je kan je workspace of environment ook leeg maken met het “bezemsteel icoon” onder de tab Environment in hulpvenster 1.

Nog een handige tip: *RStudio* ondersteunt code completion. Typ in je script of in de console `lo` en druk dan op je toetsenbord op tab-toets. Je krijgt nu een lijst met alle commando's die in *R* gekend zijn en beginnen met `lo`. Navigeer met de pijltjestoetsen totdat je de commando's over logaritmen hebt gevonden.

3 Werken met data

We bekijken hoe we een dataset kunnen importeren en zelf maken in *RStudio* zodat we deze kunnen onderzoeken. Je vindt de datasets die je voor de computeroefenzittingen nodig hebt terug op blackboard. In dit document werken we met de dataset *resultaten.txt*.

3.1 Dataset importeren

We tonen hoe je simpele tekst bestanden kunt importeren zoals *.txt*, *.csv* en *.dat* bestanden. Ga naar de tab “Environment” in hulpvenster 1 en onder Import Dataset kies je: *From Text (base)*.... Browse naar waar je *resultaten.txt* hebt opgeslagen en open het bestand.

In het pop-up scherm kies je links bovenaan de naam die je in *RStudio* aan de dataset wilt geven. Standaard vult *RStudio* hier de naam van de file die je inleest in, dus hier is dat **resultaten**. Je kan in dit venster ook selecteren hoe het input bestand is opgebouwd. De kolommen kunnen een naam hebben gekregen of niet. Afhankelijk hiervan selecteer je Heading *Yes* of *No*. In dit geval kiezen we voor *Yes*. Je moet ook aangeven hoe de verschillende entries van elkaar gescheiden worden. In dit geval werd er gebruikt gemaakt van *Whitespace*. Ook moet je kiezen of decimale getallen met een *punt* of een *komma* worden genoteerd. Aangezien er geen kommagetallen in de dataset staan is deze keuze nu minder belangrijk.

Om data uit Excel bestanden te importeren, dus bestanden die eindigen in *.xls* of *.xlsx*, heb je een extra package nodig, maar dit bespreken we hier niet. Voor andere ingewikkeldere datatypes is dit ook meestal nodig.

3.2 Eigen dataset ingeven

Je kan ook een eigen dataset ingeven. Dit gebeurt door het ingeven van verschillende commando's in een script en deze uit te voeren via de console.

We hebben de volgende dataset:

Var1 = Kleur	Var2 = Geslacht	Var3 = Aantal
D	M	9
D	V	14
L	M	12
L	V	11
R	M	3
R	V	7

Om deze in R te zetten voeren we de volgende code uit:

```
Kleur = c("D","D","L","L","R","R")
Geslacht = c("M","V","M","V","M","V")
Aantal = c(9,14,12,11,3,7)
Dataset = data.frame(Kleur,Geslacht,Aantal)
Dataset
```

Kleur, Geslacht, Aantal en Dataset zijn de namen van de variabelen waarin de gegevens bewaard worden. Je kan deze gerust anders noemen, bijvoorbeeld

```
x = c("D","D","L","L","R","R")
y = c("M","V","M","V","M","V")
z = c(9,14,12,11,3,7)
d = data.frame(x,y,z)
d
```

Met het commando `c` maak je een vector, het voegt verschillende getallen of objecten samen in één object. `data.frame` voegt verschillende vectoren samen tot een matrix met als object type een dataset. Merk op dat als je een naam hergebruikt, de vorige inhoud ervan verloren gaat.

Goed om weten: *R* is hoofdlettergevoelig, dit wil zeggen dat `Dataset` niet hetzelfde betekent als `dataset`. Dit laatste commando zal een foutmelding opleveren wanneer je het uitvoert in *R*.

4 Beschrijven van data

Na het inladen van de dataset *resultaten.txt* kunnen we de data visualiseren en beschrijven met verschillende commando's in *R*.

4.1 Histogram

Een **histogram** van de variabele `LENGTE` maak je als volgt:

```
lengte=resultaten$lengte
hist(lengte, main="Histogram van lengte studenten", xlab="lengte (in cm)",
     ylab= "Absolute frequentie", ylim=c(0,20))
```

waarbij `lengte` de variabele is waarvan je het histogram wilt tekenen, `xlab` geeft een naam aan de X-as, `ylab` geeft een naam aan de Y-as, `main` geeft een titel aan de grafiek en `ylim=c(y1,y2)` bepaalt de grenzen op de Y-as.

Met *RStudio* kan je ook een **dichtheidshistogram** tekenen door `freq=FALSE` als argument mee te geven.

```
hist(lengte,main="Dichtheidshistogram van lengte studenten",
      xlab="lengte (in cm)", ylab= "Relatieve frequentie", freq=FALSE)
```

De totale oppervlakte is gelijk aan 1 bij een dichtheidshistogram.

4.2 Kenmerken van locatie en schaal

Het **steekproefgemiddelde** gebruikt men vaak om de locatie (centrum) van een verdeling aan te duiden. De definitie van het steekproefgemiddelde is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

met x_i , $i = 1, \dots, n$ de datapunten.

Om het gemiddelde van de lengtes van de jongens en van de meisjes uit de dataset *resultaten.txt* te berekenen, moet je eerst de gegevens opdelen in de lengtes van de jongens en die van de meisjes met behulp van de commando's:

```
lengteV=lengte[resultaten$geslacht=="V"]
lengteM=lengte[resultaten$geslacht=="M"]
```

Om nu het gemiddelde te berekenen van beide variabelen voeren we de volgende commando's uit:

```
mean(lengteV)
mean(lengteM)
```

De **steekproefmediaan** is gedefinieerd als

$$\text{med}(x) = \begin{cases} x_{(n+1)/2} & \text{als } n \text{ oneven} \\ \frac{x_{(n/2)} + x_{((n/2)+1)}}{2} & \text{als } n \text{ even.} \end{cases}$$

Om dit te berekenen in *R* doen we:

```
median(lengteV)
median(lengteM)
```

Merk op dat de steekproefmediaan van de lengtes van de meisjes erg weinig verschilt van het gemiddelde. Dit kan je verklaren aan de hand van het histogram van deze gegevens, dat we tekenen met het commando `hist(lengteV)`. De lengtes van de meisjes lijken vrij symmetrisch verdeeld rond de klasse $[165, 170]$. Dit verklaart waarom de mediaan zo dicht bij het gemiddelde gelegen is. Als we nu het histogram van de lengtes van de jongens tekenen door `hist(lengteM)` zien we 2 pieken. Dit kan wijzen op een bimodale verdeling.

De definitie van de **empirische variantie** van een steekproef is de volgende:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

De **empirische standaardafwijking** is de vierkantswortel uit de empirische variantie. De empirische variantie en dus ook de empirische standaarddeviatie zijn erg gevoelig voor uitschieters.

We berekenen de standaarddeviatie en de variantie van de lengtes van de meisjes en controleren dat de variantie het kwadraat is van de standaarddeviatie.

```
sd(lengteV)
sd(lengteV)^2
var(lengteV)
```

De **empirische interkwartielafstand** is gedefinieerd als

$$IQR_n = \hat{Q}_n(0.75) - \hat{Q}_n(0.25)$$

Je kan narekenen dat de IQR_n voor de lengte van de meisjes 8.25 bedraagt met de code

```
IQR(lengteV)
```

Nog een andere robuuste maat voor de spreiding is de **Median Absolute Deviation** met als definitie

$$\text{mad}(x) = \text{med}|x_i - \text{med}(x_i)|.$$

In *RStudio* kan men $\text{mad}(x)$ berekenen met behulp van het commando `mad`. Als je echter `?mad` in de *R* console intypt, zie je dat in *RStudio* $\text{mad}(x)$ als volgt gedefinieerd is: $\text{mad}(x) = 1.4826 \text{med}|x_i - \text{med}(x_i)|$. We verkrijgen daarom $\text{mad}(x)$ uit de definitie zoals in de cursus door als argument `constant=1` mee te geven. De $\text{mad}(x)$ voor de lengtes van de meisjes is 4.5 en bereken je met:

```
mad(lengteV, constant=1)
```

Voor gegevens uit een symmetrische verdeling geldt dat $2 \text{mad}(x) \approx IQR_n$.

4.3 Boxplot

Een **boxplot** is een handig hulpmiddel om univariate gegevens grafisch voor te stellen. Deze toont immers veel nuttige info over de gegevens. Een boxplot van de lengtes van de meisjes teken je met het commando:

```
boxplot(lengteV)
```

Merk op dat het aannemelijk is dat de lengtes van de meisjes uit een symmetrische verdeling komen. De mediaan ligt immers in het midden van de box en de whiskers zijn ongeveer even lang. We vermoedden dit ook al op basis van het histogram. Er worden geen gegevens aangeduid als uitschieter.

In *RStudio* kan je ook boxplots tekenen voor verschillende groepen. Zo kan je bijvoorbeeld een boxplot tekenen van de lengtes van de vrouwelijke studenten en één van de mannelijke studenten:

```
boxplot(resultaten$lengte~resultaten$geslacht)
```

Na het tekenen `~` geven we aan op basis van welk criterium de gegevens opgedeeld worden in twee groepen.

5 Kansrekenen met *R*

De meeste bekende verdelingen zijn standaard gedefinieerd in *R*. Voor elk van deze verdelingen kan je 4 dingen doen. Dit word hieronder uitgelegd voor de continue uniforme verdeling:

- `runif(n,min=a,max=b)` trekt op een willekeurige manier n getallen uit een uniforme verdeling tussen a en b (`r` van random). Indien er geen minimum en maximum worden meegegeven, wordt het interval $[0, 1]$ genomen.
- `dunif(x,min=a,max=b)` berekent $f(x)$ (de dichtheidsfunctie in x , `d` van density).
- `punif(x,min=a,max=b,lower.tail=TRUE)` berekent $F(x) = P(X \leq x)$ (de verdelingsfunctie in x , `p` van probability). `lower.tail` heeft `TRUE` als default waarde. Indien je hier `FALSE` invult, wordt $P(X > x)$ berekend.

- `qunif(p,min=a,max=b,lower.tail=TRUE)` zoekt x zodat $P(X \leq x) = p$ voor $0 \leq p \leq 1$ (het p -de kwantiel, `q` van `quantile`). Indien je `FALSE` invult bij `lower.tail`, krijg je de x zodat $P(X > x) = p$ wat overeenkomt met het $1 - p$ -de kwantiel.

Merk op dat de namen van de opties (bv. `min`) niet meegegeven moeten worden zolang je de juiste volgorde aanhoudt. Dus `runif(n,1,2)` geeft het zelfde resultaat als `runif(n,min=1,max=2)`. De optie `lower.tail` heeft als default waarde `TRUE`, dus als de gebruiker niets invult voor deze optie, wordt `TRUE` gebruikt.

In de onderstaande tabel staan de commando's achtervoegsels voor enkele veelgebruikte verdelingen.

Verdeling	Commando achtervoegsel	Extra parameters
Bernoulli	<code>binom(*,1,prob)</code>	<code>prob=succeskans</code>
Binomiaal	<code>binom(*,size,prob)</code>	<code>size=aantal</code> , <code>prob=succeskans</code>
Geometrisch	<code>geom(*,prob)</code>	<code>prob=succeskans</code>
Negatief Binomiaal	<code>nbinom(*,size,prob)</code>	<code>size=# successen nodig</code> , <code>prob=succeskans</code>
Hypergeometrisch	<code>hyper(*,m,n,k)</code>	<code>m=# witte knikkers</code> , <code>n=# zwarte knikkers</code> <code>k=# trekkingen</code>
Poisson	<code>pois(*,lambda)</code>	<code>lambda=α</code>
Continu Uniform	<code>unif(*,min=a,max=b)</code>	<code>[a,b]</code> = interval
Exponentieel	<code>exp(*,rate)</code>	<code>rate=α</code> (default 1)
Univ. Normaal	<code>norm(*,mean,sd)</code>	<code>mean=μ</code> (default 0), <code>sd=σ</code> (default 1)
Chikwadraat	<code>chisq(*,df)</code>	<code>df=vrijheidsgraden</code>
Gamma	<code>gamma(*,shape,rate=1,scale)</code>	<code>scale=β</code> , <code>shape=γ</code>
t	<code>t(*,df)</code>	<code>df=vrijheidsgraden</code>
Cauchy	<code>cauchy(*)</code>	
F	<code>f(*,df1,df2)</code>	<code>df1,df2=vrijheidsgraden</code>
Lognormaal	<code>lnorm(*,meanlog,sdlog)</code>	<code>meanlog=μ</code> (default 0), <code>sdlog=σ</code> (default 1)

Voorbeelden:

- Om de dichtheidsfunctie van een normale verdeling te berekenen in x , gebruik je het commando `dnorm(x,mean,sd)`
- Het p -de kwantiel van een F -verdeling bereken je via `qf(p,df1,df2)`.
- Om n observaties van een exponentiële verdeling te trekken gebruik je het commando `rexp(n,rate)`

6 Grafieken in R

We kunnen de dichtheids- en verdelingsfuncties van variabelen ook in een grafiek zetten met `R`. Om de grafiek van een functie te maken, moet je eerst twee vectoren van gelijke lengte definiëren die je ten opzichte van elkaar wil uitzetten. Stel dat we dichtheidsfunctie van de standaard normale verdeling willen plotten voor het interval $[-3,3]$, dan kunnen we dit doen met volgende code:

```
x=seq(-3,3,0.01)
y=dnorm(x)
plot(x,y,type="l",xlab="x",ylab=expression(phi(x)),
     main="Dichtheidsfunctie standaard normale verdeling")
```

Het `seq` (sequence) commando creëert een vector x met als eerste element -3 , laatste element 3 en met tussenstappen van 0.01 . Indien we tussenstappen van 1 wilden, konden we ook de code `x=-3:3` gebruiken.

De eerste parameter van het commando `plot` (`x`) wordt uitgezet op de X -as en de tweede (`y`) op de Y -as. De volgorde van de overige parameters is onbelangrijk: `type` specificeert het soort plot (punten "p" (default), lijnen "l", ...), `xlab` geeft een naam aan de X -as, `ylab` geeft een naam aan de Y -as en `main` geeft een titel aan de grafiek. Grafische parameters moeten steeds tussen aanhalingstekens staan, tenzij anders bepaald.

Enkele andere interessante plotparameters zijn `xlim=c(min,max)`, `ylim=c(min,max)` (grenzen voor de X - en de Y -as), `pch=1` (punttype), `lty=1` (lijntype) en `col=1` (kleur). Zie `?par` voor een overzicht van de mogelijkheden.

Telkens wanneer je een `plot` commando uitvoert, zal dit de vorige plot overschrijven. Indien je meerdere grafieken op dezelfde figuur wil, gebruik je best het commando `lines` (lijngrafiek) of `points` (punten). Deze doen hetzelfde als `plot`, maar overschrijven de vorige plot niet. Je kan analoog als bij het `plot` commando grafische parameters zoals `col`, `lty` (voor `lines`) of `pch` (voor `points`) meegeven, maar je kan reeds aanwezige gegevens hiermee niet aanpassen of bijvullen (`xlim`, `xlab`, `main`, ... werken dus niet).

De volgende code tekent de cumulatieve verdelingsfuncties van $Y_1 \sim \mathcal{N}(0, 1)$ en $Y_2 \sim \mathcal{N}(0, 2)$ op het interval $[-4, 4]$ in dezelfde figuur.

```
x=seq(-4,4,0.01)
y1=pnorm(x)
y2=pnorm(x,0,sqrt(2))
plot(x,y1,type="l",xlab="x",ylab="F(x)",
     main = "Verdelingsfuncties normale verdelingen")
lines(x,y2,col="red",lty=3)
```

Merk op: Om de cumulatieve dichtheid van Y_2 te berekenen gebruiken we het commando `pnorm(x,0,sqrt(2))`, in R code moeten we dus de standaarddeviatie meegeven om de juiste dichtheid te berekenen, niet de variantie die je ziet staan in $\mathcal{N}(0, 2)$!

Dit is een beknopt overzicht van de belangrijkste dingen in R die we in de volgende computeroefenzittingen gaan gebruiken. Het is bewust kort gehouden, zodat je snel een overzicht hebt zonder te veel leeswerk. Probeer de commando's uit, experimenteer ermee en los oefeningen uit eerdere oefenzittingen op in R. Zoek mogelijks ook online of op andere manieren naar extra info over R.